

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.3

INHERITANCE AND POLYMORPHISM

 Copy as Markdown

Inheritance and polymorphism

Model inheritance lets you define fields once in a parent class and share them across child classes with `extends`, or model a closed set of variants with `sealed`. Polymorphism builds on this: endpoints can take and return the parent type, and the actual subtype survives the round trip between client and server. When old and new app versions run side by side, unknown subtypes can fall back to their base class. See [Handling unknown class names](#) and the [backward compatibility](#) rules.

Inheritance

Serverpod models support inheritance, which allows you to define class hierarchies that share fields between parent and child classes. Common fields, such as audit timestamps, are defined once in a parent class, and every child gets them. Generated classes will maintain the same type hierarchy as the model files.

Extending a class

To inherit from a class, use the `extends` keyword in your model files, as shown below:

```
class: ParentClass
fields:
  name: String
```

```
class: ChildClass
extends: ParentClass
fields:
  age: int
```

This will generate a class with both `name` and `age` fields.

In the generated class, inherited fields come first. To change that for a field, mark it with the `tail` keyword. See [Ordering inherited fields](#).



Ask AI

Inheritance on table models

Inheritance can also be used with table models. However, a class cannot define a `table` if another class above it in its inheritance chain already has one. The table can be placed at any level in the chain (top, middle or bottom). Separate branches of a hierarchy can each define their own table, so a parent without a table can have several child classes that each have one, as in the [polymorphism example](#) below.

INFO

This is a current limitation due to the parent class implementing the `table` getter and other table-related fields, so classes that `extends` the parent cannot override such properties with different types.

When a class in the hierarchy has a table, all inherited fields are stored as columns in that table. The `id` field is automatically added to table classes and inherited by child classes. You can customize the `id` type in a parent class, and children will inherit it.

A common use case for inheritance on table models is to have a base class that defines a custom `id` type, audit fields and other common properties that must be present on several table models. Below is an example:

```
class: BaseClass
fields:
  id: UuidValue?, defaultPersist=random_v7
  createdAt: DateTime, default=now
  updatedAt: DateTime, default=now
```

```
class: ChildClass
extends: BaseClass
table: child_table
fields:
  name: String

indexes:
  created_at_index:
    fields: createdAt # Index on inherited field
```

As the example shows, indexes can be defined on inherited fields in a child class that has a table. Relations also work normally with inherited table classes.

Restrictions

- If a parent class is marked as `serverOnly`, all child classes must also be marked as `serverOnly`. A non-serverOnly class cannot extend a serverOnly class, but a serverOnly child can extend a non-serverOnly parent.
- You can only extend classes from your own project or from [shared packages](#), not from modules.

- Child classes cannot redefine fields that exist in parent classes.

To use a base model that is shared between server and client and extend it on the server with a table, see [Shared packages](#).

Ordering inherited fields

Inherited fields are ordered from the root class down to the child class. Since a base class is declared first, its audit fields end up in front of the fields that describe the child. Mark a field with the `tail` keyword to push it behind every normal field in the hierarchy instead.

Add `tail` after the field type and any other keywords:

```
class: BaseEntity
fields:
  ownerId: UuidValue
  createdAt: DateTime, default=now, tail
  updatedAt: DateTime, default=now, tail
```

Tail ordering applies across the whole hierarchy. Normal fields keep their root-to-child order, and the tail fields follow, starting from the most specific child and working back toward the root parent:

```
class: Document
extends: BaseEntity
fields:
  authorName: String
  archivedAt: DateTime?, tail
```

```
class: Article
extends: Document
fields:
  title: String
  publishedAt: DateTime?, tail
```

The generated `Article` class orders its fields like this:

1. `ownerId`, a normal field from the root class.
2. `authorName`, a normal field from the parent class.
3. `title`, a normal field from the child class.
4. `publishedAt`, a tail field from the child class.
5. `archivedAt`, a tail field from the parent class.
6. `createdAt` and `updatedAt`, tail fields from the root class, in the order they are declared.

The same order applies to the generated constructor parameters and the serialized output. For table models it also determines the generated table definition, where the primary `id` column always comes first. The `tail` keyword cannot be used on `id`.

Adding or removing `tail` only affects generated code. It does not rename a database column, and it does not produce a migration that reorders the columns of an existing table.

Sealed classes

In addition to the `extends` keyword, you can also use the `sealed` keyword to create sealed class hierarchies. The compiler then knows all subclasses, so a `switch` over the model is exhaustive and every possible case must be handled.

INFO

If a class is sealed, it cannot have a table property. This is because a sealed class is abstract and cannot be instantiated, so it cannot represent a table row.

```
class: ParentClass
sealed: true
fields:
  name: String
```

```
class: ChildClass
extends: ParentClass
fields:
  age: int
```

This generates classes equivalent to:

```
sealed class ParentClass {
  String name;
}

class ChildClass extends ParentClass {
  int age;
}
```

Polymorphism

Serverpod supports polymorphism for models that use inheritance. When you define a class hierarchy you can use parent types as parameters and return types in your endpoints, and Serverpod will automatically serialize and deserialize the correct subtype based on the runtime type.

Below is an example of a polymorphic model hierarchy. The `EmailNotification` and `SMSNotification` classes extend the `Notification` sealed class. Each notification type has its own table and specific fields for delivery. Note that it is not possible to define relations to the `Notification` class, since it does not have a table.

```
class: Notification
sealed: true
fields:
  title: String
  message: String
  createdAt: DateTime, default=now
  sentAt: DateTime?
```

```
class: EmailNotification
extends: Notification
table: email_notification
fields:
  recipientEmail: String
  subject: String
```

```
class: SMSNotification
extends: Notification
table: sms_notification
fields:
  phoneNumber: String
  provider: String?
```

Using polymorphic types in endpoints

Polymorphic types can be used as parameters and return types in endpoint methods and streaming endpoints. The runtime type is preserved through serialization and deserialization, including inside Lists, Maps, Sets, Records, and nullable contexts:

```
class NotificationEndpoint extends Endpoint {
  Future<Notification> sendNotification(
    Session session, {
      required Notification notification,
    }) async {
    final sentNotification = switch (notification) {
      EmailNotification email => await _sendEmail(session, email),
      SMSNotification sms => await _sendSMS(session, sms),
    };

    return sentNotification.copyWith(sentAt: DateTime.now());
  }

  /// Save to database and send email
  Future<EmailNotification> _sendEmail(
    Session session,
```

```

    EmailNotification notification,
) async {
    final saved = await EmailNotification.db.insertRow(session, notification);
    // ... email sending logic
    return saved;
}

/// Save to database and send SMS
Future<SMSNotification> _sendSMS(
    Session session,
    SMSNotification notification,
) async {
    final saved = await SMSNotification.db.insertRow(session, notification);
    // ... SMS sending logic
    return saved;
}
}

```

Handling unknown class names

When deserializing polymorphic types, Serverpod uses the class name encoded in the serialized data to determine which concrete subtype to instantiate. However, there are situations where the class name in the incoming data may not correspond to any known class on the server or client:

- An older client is sending data with a class that no longer exists on the server.
- An older client is receiving data from a class that was recently added on the server.
- A newer client is sending data with a class that hasn't been deployed to the server yet.

If the missing class is a subclass of a known class, Serverpod will try to deserialize the model as the known class. This makes it safe to replace base classes with subclasses on endpoints without breaking [backward compatibility](#).

INFO

This will only work for non-streaming endpoints. Streaming endpoints will always throw an exception if the class name is not known.

Example scenario

Consider a notification system where you initially had the following type:

```

# Notification class that was not originally inherited.
class: Notification
fields:
  title: String
  message: String

```

If you later add another notification type to the server, older clients will deserialize it as the base `Notification` class instead of throwing an exception.

```
# New class that is not yet available on some older clients.  
class: EmailNotification  
extends: Notification  
fields:  
  recipientEmail: String
```

 WARNING

Note that this behavior does not apply if the base class is a `sealed` class, since it is not possible to instantiate a `sealed` class. In this case, an exception will be thrown.